

Lesson 6: Data Wrangling and Aggregation

Python Essentials — Code the Dream

Game Plan for Today

Section	Time	What We'll Do
Warm-Up	5 min	Predict groupby output
I Do	10–15 min	Selection, groupby, pivot tables, merge, apply
We Do	15–20 min	Wrangle football results together
You Do	5–10 min	Filtering and aggregation challenge
Wrap-Up	5 min	Assignment preview & Kaggle intro

Warm-Up (5 min)

What Does This Code Produce?

```
import pandas as pd

data = {"Team": ["A", "B", "A", "B", "A"],
        "Points": [10, 20, 30, 40, 50]}
df = pd.DataFrame(data)

result = df.groupby("Team")["Points"].mean()
print(result)
```

Predict the output, then share in the chat.

I Do: Data Selection Review

Four ways to access data:

Method	Selects by	Example
<code>df["col"]</code>	Column name	<code>df["Age"]</code>
<code>df.loc[row, col]</code>	Labels (inclusive)	<code>df.loc[0:2, "Name"]</code>
<code>df.iloc[row, col]</code>	Position (exclusive)	<code>df.iloc[:2]</code>
<code>df[condition]</code>	Filter	<code>df[df["Age"] > 25]</code>

Important: `.loc[0:2]` includes row 2. `.iloc[:2]` does NOT.

```
# Combine conditions with & (not "and")
df[(df["Age"] > 25) & (df["Salary"] > 50000)]
```

I Do: `groupby()` — Split, Apply, Combine

`groupby()` is the most powerful aggregation tool in Pandas:

```
# Group by category, sum the values
df.groupby("Category")["Sales"].sum()

# Multiple aggregations at once
df.groupby("Category").agg({
    "Sales": ["sum", "mean", "count"]
})
```

The pattern:

1. **Split** the data into groups
2. **Apply** a function to each group
3. **Combine** the results

Think of it like SQL's `GROUP BY` — if you know SQL, this is the same idea.

I Do: Pivot Tables

Pivot tables reshape data for cross-tab reporting:

```
pd.pivot_table(  
    sales,  
    index="Product",          # Row labels  
    columns="Region",         # Column headers  
    values="Revenue",         # What to aggregate  
    aggfunc="sum",            # How to aggregate  
    fill_value=0,             # Replace NaN  
)
```

I Do: Pivot Tables (Cont.)

Result looks like a spreadsheet summary:

Region	East	West
Product		
Dooickey	4803	281024
Gizmo	37500	86006
Widget	13002	24014

Pivot tables = `groupby()` with a nicer layout.

I Do: Merging DataFrames

`merge()` combines DataFrames on shared columns — like SQL JOINS:

```
df1 = pd.DataFrame({"ID": [1, 2, 3], "Name": ["A", "B", "C"]})
df2 = pd.DataFrame({"ID": [2, 3, 4], "Score": [85, 92, 88]})

# Inner: only matching rows
pd.merge(df1, df2, on="ID", how="inner")

# Left: keep all from df1
pd.merge(df1, df2, on="ID", how="left")

# Outer: keep everything
pd.merge(df1, df2, on="ID", how="outer")
```


I Do: Merging DataFrames (Cont.)

Join Type	Keeps
inner	Only matching rows
left	All left + matching right
right	All right + matching left
outer	Everything from both

I Do: `apply()` for Complex Logic

When simple column math isn't enough, use `apply()` :

```
def calculate_bonus(row):  
    if row["Sales"] > 100000:  
        return row["Sales"] * 0.10  
    else:  
        return row["Sales"] * 0.05  
  
df["Bonus"] = df.apply(calculate_bonus, axis=1)
```

Key details:

- `axis=1` means "apply to each **row**"
- The function receives the entire row as a Series
- Access columns with `row["ColumnName"]`

Use `apply()` when you need to combine multiple columns with if/else logic.

I Do: Useful Utilities

Rename columns:

```
df.rename(columns={"Age": "Employee Age"}, inplace=True)
```

Sort by values:

```
df.sort_values(by="Salary", ascending=False)
```

Reset index after sorting:

```
df.reset_index(drop=True, inplace=True)
```

Transform a column:

```
df["Salary"] = df["Salary"] * 1.10 # 10% raise
```

We Do: Wrangling Football Results

The assignment asks you to find the worst defensive teams. Let's build it.

Step 1: Load the data and preview:

```
import pandas as pd

football = pd.read_csv("results.csv")
print(football.head())
```

	date	home_team	away_team	home_score	away_score
0	1872-11-30	Scotland	England	0	0
1	1873-03-08	England	Scotland	4	2
...					

Problem: Each game has a home team and away team. We need results **per team**.

We Do: Reshaping the Data

Step 2: Create two views — one for home, one for away:

```
# Select columns we need
results_1 = football[["home_team", "away_team",
                      "home_score", "away_score"]]

# Home team perspective
results_home = results_1.rename(columns={
    "home_team": "team", "away_team": "opponent",
    "home_score": "points_for", "away_score": "points_against"
})

# Away team perspective
results_away = results_1.rename(columns={
    "away_team": "team", "home_team": "opponent",
    "away_score": "points_for", "home_score": "points_against"
})
```

Why two DataFrames? Each game needs to appear twice — once per team.

We Do: Combine and Aggregate

Step 3: Stack them and find the worst defenses:

```
# Combine both perspectives
all_results = pd.concat([results_home, results_away],
                        ignore_index=True)

# Average points allowed per game, per team
points_against = (all_results
                  .groupby("team")["points_against"]
                  .mean()
                  .sort_values(ascending=False))

print(points_against.head(10))
```

Discussion: What does a high `points_against` mean?

These are the teams that allow the most goals per game — the worst defenses.

You Do: Filtering and Aggregation Challenge (5 min)

Given this DataFrame:

```
df = pd.DataFrame({  
    "Department": ["Sales", "Eng", "Sales", "Eng", "HR"],  
    "Employee": ["Alice", "Bob", "Carol", "Dave", "Eve"],  
    "Salary": [60000, 90000, 65000, 85000, 55000]  
})
```

Write code to:

1. Filter for employees with `Salary > 60000`
2. Group by `Department` and get the `mean` salary
3. Sort the result in descending order

Bonus: What's the total salary for the "Eng" department?

Assignment Preview

This week you work in a **Kaggle notebook** for the first time!

Tasks	What You'll Do
1	Data selection (columns, slicing, iloc)
2	Aggregation with groupby + agg
3	Pivot tables on ecommerce dataset
4	apply() to flag matching job listings
5	Merging DataFrames (outer join, fill NaN)
6–10	Filtering, sorting, renaming, transforms, concat
11–12	Football results wrangling (we just did this!)
13	Continue your capstone project

Kaggle Notebook Tips

First time on Kaggle? Here's the workflow:

1. Create a notebook → add Markdown + Code cells
2. **Add Input** (upper right) → search for datasets
3. Run the first cell to find file paths (`os.walk`)
4. Code cells share variables — run them in order
5. **Sessions time out** — use "Run All" to restore state

Important: If your notebook times out (you walk away), click **Run All** when you come back — your variables will be gone otherwise.

Assignment Tips

Task 3 (pivot tables): The `Purchase_Amount` column is a string with a `$` sign. Strip it first:

```
df["Purchase_Amount"] = df["Purchase_Amount"]  
    .str.replace("$", "").astype(float)
```

Task 5 (merge): Use `suffixes=["_left", "_right"]` and `np.where()` to combine Age columns.

Task 11 (football): We just built this together — follow the same pattern!

Wrap-Up

Today we covered:

- Data selection with `loc[]`, `iloc[]`, and boolean filters
- Aggregation with `groupby()` and `agg()`
- Pivot tables for cross-tab summaries
- Merging DataFrames with inner, outer, left, and right joins
- `apply()` for row-wise logic across multiple columns
- Utilities: rename, sort, reset index, transform

Quick check: When would you use `merge()` vs `concat()` ?

Getting Help This Week

- **Kaggle questions?** Check the Kaggle documentation or ask in #discussion
- The football wrangling (Tasks 11–12) uses everything from this lesson
- **Capstone progress:** This week add wrangling, aggregations, and 2+ engineered features
- Post questions in **#discussion** on Slack
- Book a **1:1 mentor session** for merge or pivot table help

Next week: Introduction to Data Visualization — making your data visual!

Data Wrangled!

From raw data to insights — one groupby at a time.

See you next session!